

Bad Smells

Thiago Andrade Ramalho

808904

01.11.2025

Teste de Software

Análise de Smells

O código reutiliza as variáveis para gerar em formato HTML e CSV, e os tipos de user.role, dentro de várias verificações do teste. Dessa forma contraria o padrão aberto/fechado, pois cada novo formato ou papel demanda alterações dispersas pelo código, elevando o risco de inconsistências e dificultando a testabilidade. Isso faz com que acople indevidamente regras de negócio à camada de apresentação, reduz coesão e encarece a evolução.

Números Mágicos. Os valores de 500 e 1000 para representar as regras de acesso e prioridade faz com que dificulte a interpretação do que esses valores representam, ou confundilos ou simplesmente esquecer os respectivos contextos. Como esses numeros estão distribuídos em vários if, é possível esquecer ou passar batido, se caso ocorrer uma manutenção, de alterar o limite em todos os lugares, com isso, esse trecho se enquadra em um conceito de smell.

Duplicação de código, pois houve muitos trechos em que eram repetidos e representavam a mesma ideia, por exemplo para a lógica do cabeçalho é repetido tanto para o CSV quanto para o HTML, além de serem repetidos para o cabeçalho, para o corpo e praticamente para o rodapé também, multiplicando o esforço de manutenção. Fora que dentro do loop, a sintaxe 'total += item.value', aparece em vários ramos, embora a soma seja a mesma ação em todos os casos, assim, com a repetição, pode ocorrer chances maiores de inconsistências.

Relatório da Ferramenta

```
PS C:\Users\Thiago Andrade\Documents\GitHub\bad-smells-js-refactoring> npx eslint src/

C:\Users\Thiago Andrade\Documents\GitHub\bad-smells-js-refactoring\src\ReportGenerator.js
  11:5   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5 allowed  sonarjs/cognitive-com
plexity
  43:16  error  Merge this if statement with the nested one                                       sonarjs/no-collapsibl
e-if

✖ 2 problems (2 errors, 0 warnings)
```

O primeiro erro exibido no terminal, foi cognitive-complexity, o que significa que a função possui vários caminhos, aumentando a dificuldade de entender o fluxo e de cobrir tudo com testes. O segundo foi no-collapsible-if, que representa condições de if que poderiam ter sido feitas juntas, minimizando o tamanho do código e facilitando o entendimento e a manutenção do teste. Com esses reports de erros/problemas trazidos pelo eslint-plugin-sonarjs, faz com torne objetivo o que costuma passar despercebido na leitura, mostrando onde a lógica está muito mais confusa

e onde alterar. Ficando mais fácil refatorar para um código mais coeso, modular e extensível, melhorando a manutenção e a qualidade dos testes.

Processo de Refatoração:

```
26 // --- Seção do Corpo (Alta Complexidade) ---
27 for (const item of items) {
28   if (user.role === 'ADMIN') {
29     // Admins veem todos os itens
30     if (item.value > 1000) {
31       // Lógica bônus para admins
32       item.priority = true;
33     }
34
35     if (reportType === 'CSV') {
36       report += `${item.id},${item.name},${item.value},${user.name}\n`;
37       total += item.value;
38     } else if (reportType === 'HTML') {
39       const style = item.priority ? 'style="font-weight:bold;"' : '';
40       report += `|
|  |

```

```

53   generateReport(reportType, user, items) {
54       const fmt = this._getFormatter(reportType);
55       const policy = this._getPolicy(user.role);
56
57       let total = 0;
58       const parts = [];
59
60       parts.push(fmt.header(user));
61
62       for (const item of items) {
63           if (!policy.canSee(item)) continue;
64           const meta = policy.meta(item);
65           parts.push(fmt.row(item, meta, user));
66           total += item.value;
67       }
68
69       parts.push(fmt.footer(total));
70       return parts.join('').trim();
71   }

```

Foi trocado os if/else para um objeto que ira verificar se o formato é CSV ou HTML, e um outro objeto para as regras de user role, com isso nao é mais necessário testar as condições o tempo todo. A verificação de visibilidade também foi separada, para o metodo canSee, assim o fluxo fica mais simples, pois é renderizado e somado o total sempre no mesmo lugar, sem ficar repetindo o total += item.value em cada ramo. Também foi alterado o item.priority = ..., agora é calculado um meta.priority so para a renderização. Dessa forma o código facilita a leitura e o teste.

```

PS C:\Users\Thiago Andrade\Documents\GitHub\bad-smells-js-refactoring> npx eslint .\src\ReportGenerator.refactored.js
PS C:\Users\Thiago Andrade\Documents\GitHub\bad-smells-js-refactoring>

```

Conclusão

Testes garantem que nada quebre enquanto refatora, pois primeiro escreve testes que capturam o comportamento atual, mesmo mudando a estrutura interna, seja dividindo métodos grandes, trocando if/else por outras estratégias, removendo repetições, caso algo não funcione, os testes irão relatar na hora, dando uma maior segurança para melhorar o design. Em conjunto, reduzir bad smells deixa o código mais claro e seguro, tendo menos condicionais juntas e com isso uma árvore de caminhos menores; tirar números mágicos e duplicação

aumenta a coesão e diminui pontos de erro. Pois evitar efeitos colaterais faz com que seja mais fácil de prever e testar, resultando em um sistema mais estável, mais barato de manter e mais fácil de evoluir, e até os próprios testes ficam mais simples, porque validam o que realmente importa o comportamento que o usuário vê.